

Module 10 Lab Session 3: Full-Stack Integration, Error Handling, and Optimistic Rollback

Module	M10 Full-Stack Integration
Session	3 of 3
Exercises	Exercises 3 and 4

Story Thread

Liya tries to enroll in *Database Internals*, but the course reaches maximum capacity right as her request hits the server. The .NET API returns HTTP status 400 Bad Request containing an RFC 7807 ProblemDetails JSON body with a specific message: *"Course DB-401 is at maximum capacity."*

Without structured error interceptors, Angular might show a generic "Http failure response for (unknown url): 0 Unknown Error", leaving Liya guessing what went wrong.

Meanwhile, Abeba deletes an old course from the catalog. In modern applications, users expect instant UI feedback—they do not want to wait 800ms for a network round trip. But if Abeba deletes a course that still has active enrollments, the backend rejects the deletion (409 Conflict). How do we give users instant UI feedback while reliably restoring the original state if the server rejects the action?

In this final integration session, you complete the full-stack loop: 1. **Server Side ProblemDetails & Error Interceptor:** Configure .NET 10 to emit RFC 7807 standard error payloads, parse them centrally in Angular, and handle expired sessions (401 Unauthorized) seamlessly. 2. **Optimistic UI Deletion with Automatic Rollback:** Instantly mutate local store state for instant visual feedback, while snapshotting state to roll back cleanly if server validation fails. 3. **End-to-End Integration Sprint:** Bring CORS policies, cookies, XSRF headers, error interceptors, and real-time SignalR updates together into a complete, working full-stack system!

Exercise 3: Full-Stack Integration, Error Handling, and Optimistic Rollback

Part A: Server-Side ProblemDetails & Angular Error Interceptor

First, let's ensure our .NET API formats all exception and validation errors as RFC 7807 ProblemDetails JSON objects, then write an Angular functional `HttpInterceptor` to catch HTTP errors globally.

1. Open `Program.cs` in your Web API project. Register `ProblemDetails` middleware services:

```
// Add RFC 7807 ProblemDetails support to the DI container  
builder.Services.AddProblemDetails();
```

2. Scroll down to the middleware pipeline (after `builder.Build()`) and enable the status code pages middleware:

```
app.UseStatusCodePages(); // Converts 4xx/5xx responses into standard ProblemDetails payloads
```

3. Next, create `src/app/interceptors/error.interceptor.ts` in your Angular project:

```
export const errorInterceptor: HttpInterceptorFn = (req, next) => {  
  {  
    const router = inject(Router);  
  
    return next(req).pipe(  
      catchError((err: HttpResponse) => {  
        // Extract C# RFC 7807 ProblemDetails detail property  
        const detailMessage = err.error?.detail ?? 'A system error occurred. Please try again.';  
  
        if (err.status === 401) {  
          // Redirect expired or unauthenticated sessions back to login  
          router.navigate(['/login']);  
        } else {  
          // Surface structured error to developer console / UI notification  
          console.error('API Error Response:', detailMessage);  
        }  
  
        return throwError(() => err);  
      })  
    )  
  }  
}
```

```
);  
};
```

4. Register `errorInterceptor` in `src/app/app.config.ts` alongside your `credentialsInterceptor`:

```
import { credentialsInterceptor } from './interceptors/credentials.interceptor';  
import { errorInterceptor } from './interceptors/error.interceptor';  
  
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideHttpClient(  
      withInterceptors([credentialsInterceptor, errorInterceptor]),  
      withXsrfConfiguration({  
        cookieName: 'XSRF-TOKEN',  
        headerName: 'X-XSRF-TOKEN',  
      })  
    ]  
  }  
};
```

Part B: Optimistic UI Deletion with Automatic Snapshot Rollback

Now, let's implement optimistic UI updates for course deletion. When Abeba clicks **Delete Course**, we remove the entity from our `SignalStore` immediately. If the server rejects the call (for instance, if active student enrollments exist), we restore the previous snapshot seamlessly.

1. Open `src/app/store/course.store.ts`.
2. Implement `deleteCourse` inside `withMethods`:

```
export const CourseStore = signalStore(  
  { providedIn: 'root' },  
  withMethods((store, svc = inject(CourseService)) => ({  
    deleteCourse(id: number) {  
      // 1. Take snapshot of current entities BEFORE mutating local  
      state  
      const previousSnapshot = store.entities();  
  
      // 2. Instant visual feedback – remove entity immediately from  
      local UI
```

```

        patchState(store, removeEntity(id));

        // 3. Dispatch API call to backend server
        svc.delete(id).pipe(
            catchError(err => {
                // 4. Server rejected request – restore previous snapshot
                and set error message
                patchState(store, setAllEntities(previousSnapshot));
                patchState(store, {
                    error: 'Cannot delete course: active student enrollments
exist.'
                });
                return EMPTY;
            })
        ).subscribe();
    }
    })))
);

```

Execution Order Rule: The `store.entities()` snapshot MUST be captured *before* calling `patchState(store, removeEntity(id))`. If you snapshot after `patchState`, your rollback snapshot will already be missing the deleted item!

Part C: SignalR CORS Verification

In **Module 9 Lab Session 3**, you built the `LiveSyncService` and `EnrollmentStore.listenForLiveUpdates` to stream real-time status updates over WebSockets. Now that your backend enforces a named CORS policy, let's verify that your SignalR hub allows cross-origin connections.

1. Open `Program.cs` in your Web API project.
2. Locate the SignalR hub mapping and ensure `.RequireCors("TmsClient")` is chained:

```
app.MapHub<TmsHub>("/hubs/tms").RequireCors("TmsClient");
```

3. Restart your API (`dotnet run`). SignalR WebSockets will now successfully negotiate cross-origin handshakes from <http://localhost:4200>.

Exercise 4: End-to-End Integration Sprint (Verification)

Bring all integration layers together and verify your complete full-stack application:

1. **Launch Both Servers:**
 - Start the ASP.NET Core API (dotnet run on port 5000).
 - Start the Angular dev server (ng serve on port 4200).
2. **Execute Identity Handshake:** Log in via Angular. Open DevTools -> Application -> Cookies and verify tms_auth is marked **HttpOnly**.
3. **Fetch Data with Environment Config:** Browse the catalog. Confirm CourseService retrieves data directly using environment.apiUrl.
4. **Mutate Data with XSRF Guard:** Perform an action (e.g. updating a record). Verify in the Network tab that the request carries the automatic X-XSRF-TOKEN header.
5. **Verify Real-Time SignalR Stream:** Open two browser windows. Trigger an update in Window 1 and observe Window 2 update automatically over WebSockets.
6. **Verify Error Interception:** Attempt an invalid action (e.g., enrolling in a full course). Confirm the browser console displays the C# ProblemDetails detail text.
7. **Test Optimistic Rollback:** Delete a course that has active enrollments. Confirm the card disappears instantly from the screen, then reappears a moment later when the server returns 409 Conflict.

Integration Complete!

Congratulations! You have successfully fused an enterprise Angular 22 frontend with an ASP.NET Core 10 backend.

Your application now operates under real browser security rules with named CORS policies, secure HttpOnly session cookies, XSRF protection, functional HTTP interceptors, rate-limited SignalR WebSocket updates, and robust optimistic UI mutations with automatic server rollback.